

Group Project: Airlines Management System

Stage 4 User Guide

Version: FINAL

Date: 5/8/2026

Team: Md Labeeb Hasan, Joseph Heifner, Eugene Ruiz

Course: CSCI 2210

System Overview

The SkyHigh Airlines Management System is a console-based Java application that simulates core airline operations. It follows a clean object-oriented design based on the **Stage 4 UML diagram** (in GitHub Repo). It is now integrated with the complete **graphical Java Swing application**. It provides two major user flows:

- **Customer Booking Flow** – Search flights, select seats, manage passengers, and complete payments.
- **Management Flow** – Full CRUD operations for flights, airports, staff, bookings, and system reporting.

Key design principles used:

- **Single Responsibility Principle (SRP):** Each class has one clear job (entities hold data, managers handle collections, MainTester manages UI and testing).
- **Management Classes:** AirportManager, StaffManager, BookingManager, and TicketManager coordinate collections and interactions.
- **CRC Card Approach:** Responsibilities and collaborators were clearly defined during development.
- **Light Coordinator Pattern:** Airline class only holds references to managers.

Key design achievements:

- Full GUI interface replacing the console menu.
- Centralized DataManager for persistent save/load to text files.
- Strong adherence to SRP and the final UML diagram.
- All managers and entities are fully integrated with the GUI layer.
- Defensive copies, proper encapsulation, and clean separation between presentation, business logic, and persistence.

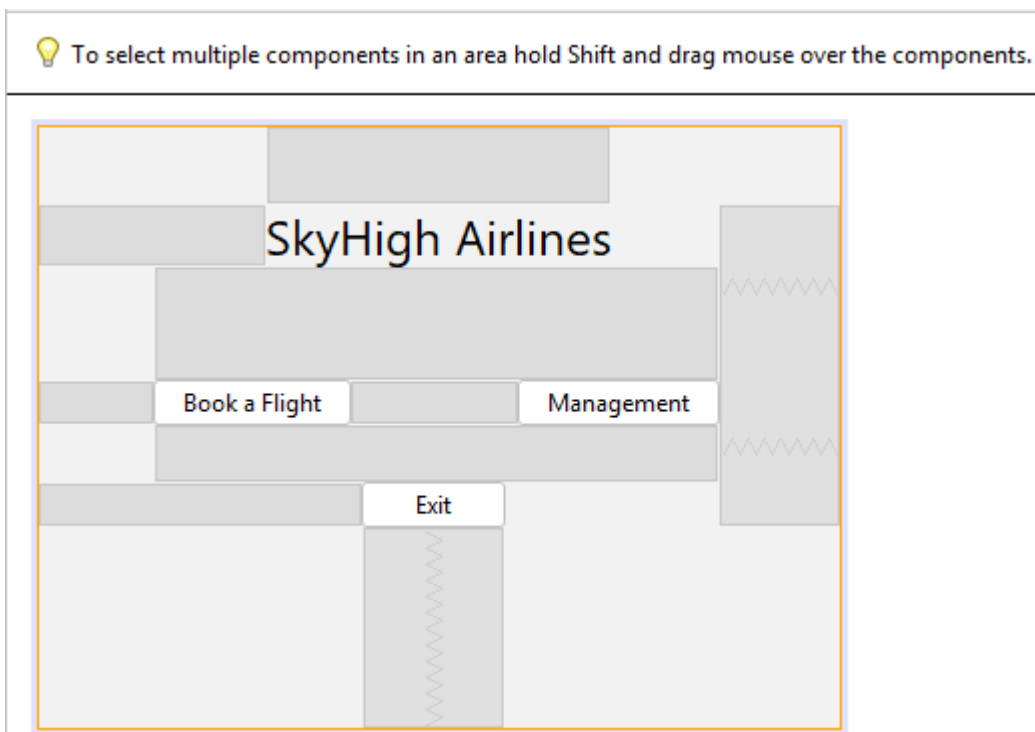
The system launches directly into the **Main Menu GUI**.

How to Run the System

1. Ensure all project classes are in the same package (AirlineSystem).
2. Compile the project.
3. Run **Main.java** (this is the official entry point).
4. The application will automatically:
 - a) Load existing data from the data/ folder (if any)
 - b) Display the **Main Menu GUI**

Note: On first run, the data/ folder will be created automatically and populated with data.

[Screenshot 1: Main Menu GUI – “Book a Flight” and “Management” buttons]



Main Menu

After startup, at the Main Menu you can choose:

- **Book a Flight** → Starts the customer booking flow
- **Management** → Opens the Management Menu for staff operations

[Screenshot 2: Main Menu GUI]



Important Sections

4.1 Booking Flow - Customer Side

1. **Flight Selection** – View available flights and select one.
2. **Seat Selection** – See real-time seat map and pick an available seat.
3. **Payment** – Enter passenger details and complete payment.

[Screenshot 3: Flight Selection GUI]

Flight	Origin	Destination	Price
AA123	PHX	LAX	199.99
DL455	LAX	LAS	149.99
UA789	DFW	ORD	299.99

Select Flight

[Screenshot 4: Seat Selection GUI with available/taken seats]

Select Your Seat

Seat	Status
1A	TAKEN
1B	AVAILABLE
1C	TAKEN
1D	AVAILABLE
1E	TAKEN
1F	AVAILABLE
2A	TAKEN
2B	AVAILABLE
2C	AVAILABLE
2D	TAKEN
2E	TAKEN
2F	AVAILABLE
3A	AVAILABLE
3B	AVAILABLE
3C	AVAILABLE
3D	AVAILABLE
3E	AVAILABLE
3F	AVAILABLE

[Screenshot 5: Payment GUI]

Payment

Total: \$199.99

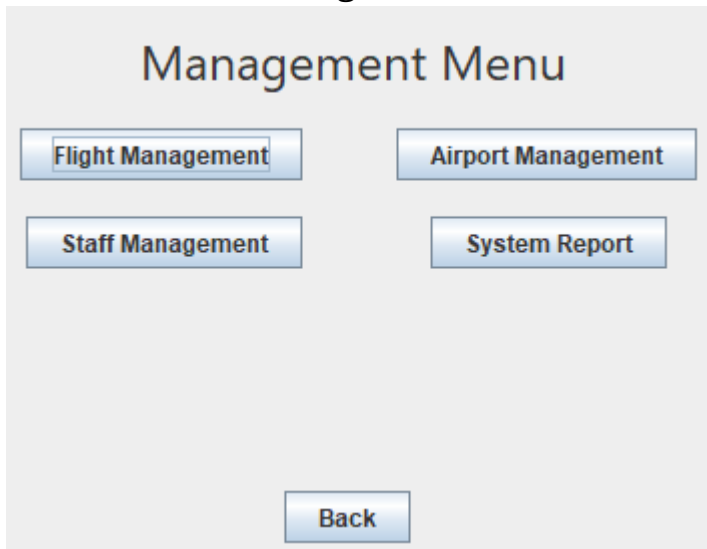
▼
Credit Card
Debit Card
Paypal

4.2 Management Menu - Staff Side

From Management you can access:

- **Flight Management** – Add, remove, update flights
- **Airport Management** – CRUD operations on airports and terminals
- **Staff Management** – Add/remove/update staff and assign to flights
- **System Report** – Comprehensive overview of all data

[Screenshot 6: Management Menu GUI]



4.3 Key Management Features

- **Flight Management GUI** – Full CRUD for flights with live table view.
- **Staff Management GUI** – Assign staff to specific flights using dropdown.
- **Booking Management GUI** – Search, update, and cancel bookings.
- **System Report GUI** – Generates a full textual report of flights, bookings, staff, and airports.

[Screenshot 7: Flight Management GUI]

The screenshot displays a web application titled "Flight Management". It features a table with four columns: "Flight #", "Origin", "Destination", and "Status". The table contains three rows of data. Below the table, there are four buttons: "Add Flight", "Remove Flight", "Update Flight", and "Back".

Flight #	Origin	Destination	Status
AA123	PHX	LAX	ON_TIME
DL455	LAX	LAS	ON_TIME
UA789	DFW	ORD	DELAYED

Buttons: Add Flight, Remove Flight, Update Flight, Back

[Screenshot 8: System Report GUI]

System Report

Ticket ID: T1778235427122 | Flight: AA123 | Seat: 1C | Status: CONFIRMED | Passenger: TE

B1778235427122 | PENDING

B1778235428601 | CONFIRMED

- Ticket ID: T1778235428601 | Flight: AA123 | Seat: 1C | Status: CONFIRMED | Passenger: TE

--- STAFF ---

S100 | Alex Morgan | Pilot

S101 | Jordan Lee | Co-Pilot

S102 | Taylor Smith | Flight Attendant

--- AIRPORTS ---

Airport[PHX] Phoenix Sky Harbor | Location: Phoenix, AZ | Terminals: 0

Airport[LAX] Los Angeles International | Location: Los Angeles, CA | Terminals: 0

Airport[LAS] Harry Reid International | Location: Las Vegas, NV | Terminals: 0

Airport[DFW] Dallas Fort Worth International | Location: Dallas, TX | Terminals: 0

Airport[ORD] O'Hare International | Location: Chicago, IL | Terminals: 0

Back

[Screenshot 9: Airport Manager GUI]

Airport Manager

Code	Name	Location
PHX	Phoenix Sky Harbor	Phoenix, AZ
LAX	Los Angeles Interna...	Los Angeles, CA
LAS	Harry Reid Internati...	Las Vegas, NV
DFW	Dallas Fort Worth In...	Dallas, TX
ORD	O'Hare International	Chicago, IL

Add Airport

Update Airport

Remove Airport

Back

[Screenshot 10: Staff Manager GUI]

Staff Management

Staff ID	Name	Contact	Role
S100	Alex Morgan	alex@skyhig...	Pilot
S101	Jordan Lee	jordan@sky...	Co-Pilot
S102	Taylor Smith	taylor@skyhi...	Flight Attend...

Add Staff

Remove Staff

Update Staff

Assign Staff To Flight

Back

AA123 | PHX -> LAX ▼

AA123 | PHX -> LAX

DL455 | LAX -> LAS

UA789 | DFW -> ORD

5.1 Data Persistence

All changes made in the GUI are automatically saved through `DataManager.saveAllData()` after major operations. Data is stored in the `data/` folder as plain text files (`airports.txt`, `flights.txt`, `staff.txt`, etc.). On startup, `DataManager.loadAllData()` restores the last saved state.

UML Foundation: The class diagram shows clear relationships (Airline owns managers; Flight links to Schedule, Aircraft, and StaffAssignment; Booking contains Tickets, etc.).

5.2 Design Principles Demonstrated

- The SkyHigh Airlines Management System demonstrates key object-oriented principles. **Inheritance** is clearly shown through the **Person** abstract class, which provides shared attributes (**name** and **contactInfo**) and behavior (**updateContactInfo()**) to both **Staff** and **Passenger** subclasses. This avoids code duplication while allowing each subclass to add specialized fields and methods (e.g., **staffID/role** for **Staff** and **passengerID/passportNumber** for **Passenger**).
- **Encapsulation** is applied throughout: all instance variables are private, accessed only via public getter/setter methods, and managers return defensive copies of collections (e.g., **TicketManager.getAllTickets()**). This protects internal data and maintains clean boundaries between classes.

System Summary

- The final system closely follows the **Stage 4 UML**.
- **GUI layer** communicates with managers via the central Airline coordinator.
- All **CRUD** operations are now fully accessible through intuitive graphical interfaces.
- **Persistence layer** (`DataManager`) ensures data survives application restarts.

Exiting the System

Close any GUI window or use the Back / Exit buttons throughout the application. The system will save all data on exit.

Thank you for using SkyHigh Airlines Management System!
